趣味指针分析

信安 2453359 李欣洋

Code如右图所示:



```cpp
/*信安 2453359 李欣洋*/
#include <iostream>
using namespace std;

/*****************************************************************************
  函数名称:
  功    能:
  输入参数:
  返 回 值:
  说    明:
*****************************************************************************/
void f_sub(int x)
{
    cout << "x = " << x << endl;
}

/*****************************************************************************
  函数名称: f_func
  功    能: 接受参数 const char *p 返回值是* 这个指针又指向一个新的函数
            这个函数接受参数int x 返回值是 void
  输入参数: const char *p
  返 回 值: void(*)(int x)
  说    明:
*****************************************************************************/
void (*f_func(const char *p))(int x)
{
    cout << "str = "<<p << endl;
    return f_sub;
}
int main()
{
    /*题目定义了f是一个数组,这个数组的元素是函数指针,(*f[3](const char *p))
    而这个函数接受的参数是 const char *p 返回值是* 这个指针又指向一个新的函数
    这个函数接受参数int x 返回值是 void*/
    void(*(*f[3])(const char *p))(int x);

    f[0] = f_func;
    f[0]("Hello")(2);
    f[0]("Tongji")(7);

    return 0;
}

```

- Step1: `void(*(*f[3])(const char *p))(int x);`

对题目中给出的复杂指针进行分析:

首先分析(*f[ ])(const char *p),f是一个数组,是一个函数指针,指向的这个函数接受的参数是const char *p

而这个函数的返回值是 * 是一个指针,即void (*)(int x),即该指针又是一个函数指针,其指向的函数接受的参数是int,返回值是void

- Step2:

```
f[0] = f_func;
```

f[0]=f_func;

让f[0]指向函数f_func

- Step3:

```
f[0]("Hello")(2);
```

```
void (*f_func(const char *p))(int x)
{
    cout << "str = "<<p << endl;
    return f_sub;
}
```

此时f[0]指向函数f_func,传入参数“Hello”,在f_func内部输出Hello,然后返回一个指针,形成void(*)(int x)的函数指针,符合f_sub,故函数返回f_sub

- Step4:

```
f[0]("Hello")(2);
```

```
void f_sub(int x)
{
    cout << "x = " << x << endl;
}
```

对于f_sub,传入参数2,输出x=2

- Step5: `f[0]("Tongji")(7);`

该行代码的执行逻辑和上一行代码相同